

Documentación Proyecto 3 PLE

Se adjunta link de las anteriores entregas en caso de que sea necesaria para la evaluación del presente proyecto: [☰ PLE Task1 Project1](#) , [☰ PLE Project 2](#)

Comentarios Entrega anterior:

“ your grammar covers all required VeriLang constructs and the BinOp type has good coverage with 18 operators including $\equiv$ and $\Rightarrow$. The Identifier lexical is correct (upper+lowercase, dashes, keyword exclusion with follow restriction), the quantifier expression with . separator works properly, and your Plugin.rsc provides VS Code integration. your test file (attempt12938472.vl) parses successfully with both $<$ (space inheritance) and $\rightarrow$ (rule rewriting).

the most critical issue is the Dominio rule: Identifier $\{ \rightarrow \}$ Identifier $\}^+$ has the separator list arguments inverted — in Rascal {Element Separator} $\}^+$ puts the element first, so this reads as "one or more $\rightarrow$ separated by Identifier" rather than "one or more Identifier separated by $\rightarrow$ ". The fix is {Identifier $\{ \rightarrow \}$ }. The AST has two problems: the field name op is reused for both UnOp and BinOp in the Expr type (violating Rascal's field uniqueness constraint), and there are case mismatches between syntax labels and AST constructors (spaceDef vs spacedef, operatorDef vs operatordef, attrItem vs attritem), Rascal is case-sensitive, so these prevent implode() from working. All binary operators share a single precedence level, which means x and y or z doesn't respect standard precedence. Main.rsc is a placeholder with no parse/transform logic, the keyword defer is missing, and there are no @category annotations for syntax highlighting.

Carla.”

- Cambios y Mejoras en el Parser y AST (Task 1 y 4)

En esta iteración se realizaron ajustes estructurales para resolver problemas de la entrega anterior y cumplir con los nuevos requisitos de tipado:

Correcciones de Estructura y Base Técnica (Feedback Iteración Anterior):

Se abordaron de manera meticulosa todas las observaciones realizadas por la monitora Carla para garantizar que la base del lenguaje fuera 100% funcional y robusta:

Corrección de la Regla de Dominio: Se corrigió la inversión de argumentos en la lista de la gramática. Se cambió {Identifier $\rightarrow$ } $\}^+$ por {Identifier $\rightarrow$ } $\}^+$ (el elemento primero, luego el separador), permitiendo que las definiciones de dominio se parseen correctamente.

Unicidad de Campos en el AST: Se resolvió el conflicto de nombres de campos en el tipo Expr. Anteriormente, el nombre op se reutilizaba para UnOp y BinOp, violando las restricciones de Rascal. Ahora cada constructor tiene identificadores de campo únicos.

Consistencia de Case-Sensitivity: Se estandarizaron los nombres de los constructores en el AST para que coincidan exactamente con las etiquetas de la sintaxis (ej. spaceDef, operatorDef, attrItem). Esto eliminó los errores de "mismatch" y permitió que la función implode() funcione satisfactoriamente.

Jerarquía de Precedencia: Se actualizó la gramática para incluir niveles de precedencia realistas (usando el operador >). Ahora, operaciones como la potencia (), seguidas de la multiplicación/división, sumas y finalmente operadores lógicos, respetan el orden estándar de evaluación.

Soporte de Palabras Clave: Se incluyó la palabra clave faltante defer en la lista de Keywords y se aseguró su exclusión en el léxico de Identifier.

Lógica en Main.rsc: El archivo Main.rsc dejó de ser un placeholder y ahora contiene la lógica completa de orquestación (verificar), conectando el Parser, el nuevo Checker y el Intérprete.

Anotaciones de Tipo:

Se actualizó la gramática (Syntax.rsc) y el AST para permitir que las estructuras de datos (defdata) incluyan un tipo explícito definido por el usuario (Int, Bool, Float, etc.), cumpliendo con el punto 4 del enunciado.

Corrección de Literales:

Se modificó la definición de los literales enteros en el AST. Anteriormente se trataban como cadenas de texto (str); ahora se transforman directamente en enteros (int) mediante la función implode. Esto simplifica la lógica del intérprete y mejora la eficiencia.

- Implementación del Sistema de Tipos (Task 3, 5 y 6)

Para la validación semántica del lenguaje, se implementó un Checker Manual basado en la Sección 8 del Tutorial de Rascal:

Justificación Técnica:

Se optó por una implementación manual debido a incompatibilidades de configuración con la librería externa TypePal en el entorno local. Esta solución garantiza el cumplimiento total de las reglas de tipado requeridas sin depender de dependencias externas que impidieran la ejecución del proyecto.

Correspondencia de Tipos (Task 5):

El Checker recorre el AST y valida que todos los operandos en expresiones matemáticas, lógicas y de comparación coincidan con los tipos esperados (ej: asegurar que los operadores aritméticos solo reciban Int).

Nueva Regla de Estructuras (Task 6):

Se definió una regla específica en `checkDataDef` que asegura que todos los elementos listados en una estructura de datos existan previamente en el entorno de variables (`defvar`). Además, se añadió una validación extra para confirmar que el tipo de cada elemento coincida con el tipo declarado para la estructura.

- Ejecución e Interpretación (Task 2)

El lenguaje ahora es capaz de ejecutar programas completos desde archivos `.vl`:

Se implementó la función `verificar` en el módulo `Main.rsc`, la cual orquesta el flujo completo: Parsing -> AST Implode -> Type Checking -> Interpretation.

Los resultados de la evaluación de reglas se muestran dinámicamente en la consola, permitiendo observar el estado de las variables y el cumplimiento de las condiciones lógicas.